

SMS++ File Format Reference

Version: 0.50

Date: 27/03/2019

Contributors:	Name	Organisation
Deliverable Leader	Antonio Frangioni	ICOOR (UniPi)
Work Package Leader	Utz-Uwe Haus	Cray
Contributing Authors	Antonio Frangioni Laura Galli Rafael Durbano Lobato Ali Ghezelsoufi Niccolò Iardella	ICOOR (UniPi)

Contents

1	Introduction	2
2	General netCDF file structure	3
3	General SMS++ file structure	4
3.1	Block description	5
3.2	Configuration description	5
3.2.1	BlockConfig description	5
3.2.2	BlockSolverConfig description	6
3.2.3	ComputeConfig description	6
4	File formats of existing :Block	8
4.1	SimpleMILPBlock	8
4.2	StructuredMILPBlock	9
4.3	MCFBlock	10
5	BlockConfig for existing :Block	11
5.1	Parameters of MCFBlock	11
6	ComputeConfig for existing :Solver	12
6.1	Standard parameters in Solver	12
6.2	Standard parameters in CDASolver	13
6.3	Standard parameters in Function	14
6.4	Standard parameters in C05Function	15
6.5	Parameters of MCFSolver<MCFC>	15
6.5.1	Parameters of MCFSolver<MCFSimplex>	16
7	Conclusion	17

1 Introduction

This document describes the format of input files for the SMS++ components of the PLAN4RES project. These are the components tasked with performing the actual solution of the different optimization problems involved in the process. The production of these files possibly requires:

- a sophisticated data extraction and preparation phase;
- a tuning phase aimed at finding appropriate parameters for the different modelling and algorithmic options of the solution process.

The description of both phases is outside of the scope of this document, which only deals with the required final form of the input.

Insomuch as the set of types of optimization problems (`Block` in SMS++ parlance) to be solved within the project, and the set of available solution methods for doing it (`Solver` in SMS++ parlance) may be expanding over time, this document is intended as dynamic: as new components will added, the document will grow to cover the corresponding input format. Modification of existing input formats will be kept to a minimum, and always agreed with the partners involved with producing them before being put in production. Appropriate versioning of both the software and this document will ensure consistency between the data preparation and data consumption processes.

The self-documenting nature of `NETCDF`, together with its portable readability and the schema definition of the various `Block`, will make it possible to re-use all or some blocks of one SMS++-generated model in other contexts.

2 General netCDF file structure

NETCDF (network Common Data Form) [1] is a set of interfaces for array-oriented data access and a freely distributed collection of data access libraries for C, Fortran, C++, Java, and other languages. The NETCDF libraries support a machine-independent format for representing scientific data. Together, the interfaces, libraries, and format support the creation, access, and sharing of scientific data.

NETCDF data is:

- Self-Describing. A NETCDF file includes information about the data it contains.
- Portable. A NETCDF file can be accessed by computers with different ways of storing integers, characters, and floating-point numbers.
- Scalable. A small subset of a large dataset may be accessed efficiently.
- Appendable. Data may be appended to a properly structured NETCDF file without copying the dataset or redefining its structure.
- Sharable. One writer and multiple readers may simultaneously access the same NETCDF file.
- Archivable. Access to all earlier forms of NETCDF data will be supported by current and future versions of the software.

For the purpose of this document, only a relatively small set of NETCDF concepts need be discussed.

- *NETCDF file*: is the basic container of NETCDF, e.g. corresponding to a standard file in the filesystem of the host machine. In the C++ interface used by SMS++ it corresponds to an object of type `netCDF::NcFile`. A typical constructor of the object requires the pathname of the file (a standard `char *`) and the mode, which can be e.g. `netCDF::NcFile::read` or `netCDF::NcFile::replace` for reading and writing, respectively.
- *NETCDF group*: a NETCDF is organised into several *groups*, which can be nested inside each other to an arbitrary degree. In the C++ interface used by SMS++, each group corresponds to an object of type `netCDF::NcGroup`; note that `netCDF::NcFile` derives from `netCDF::NcGroup`, and therefore all capabilities of the group are shared by the file (but not vice-versa). Each *group* has a unique string name by which it can be retrieved; `netCDF::NcGroup` (and, therefore, `netCDF::NcFile`) has a method `getGroup()` taking as input a `std::string` with the group name and returning an `netCDF::NcGroup` object containing the *child group* with the given name (if any) of that group.
- *NETCDF attribute*: is a simple data point specific of a `netCDF` group, having a unique string name by which it can be retrieved and a simple value (`int`, `double`, `std::string`, ...). In the C++ interface used by SMS++, it is represented by a `netCDF::NcGroupAtt` object, which can be obtained by its enclosing `netCDF::NcGroup` via the method `getAtt()` taking in input its name. Then, the method(s) `getValue()` (available with different signatures corresponding to the different supported basic data types) allows to retrieve its value.
- *NETCDF dimension*: it basically represent the different indices over which the actual data is indexed, each of which is given a name. In the C++ interface used by SMS++, it is represented by a `netCDF::NcDim` object, which can be obtained by its enclosing `netCDF::NcGroup` via the method `getDim()` taking in input its name. The dimension (a `size_t` value) is returned by the method `getSize()`.
- *NETCDF variable*: this is the container for the “complex” data of a *groups*. It has a name (`std::string`), a basic data type (`int`, `double`, `std::string`, ...) and in principle any number of *dimensions*. In the C++ interface used by SMS++, it is represented by a `netCDF::NcVar` object, which can be obtained by its enclosing `netCDF::NcGroup` via the method `getVar()` taking in input its name. Once the `netCDF::NcVar` object is obtained, individual values of the *variable* can be read by its method(s) `getVar()` (available with different signatures corresponding to the different supported basic data types), taking a `std::vector<size_t>` as long as the the number of *dimensions* of the *variable* (obtainable e.g. with its method `NgetVarDimCount()`).

The above components allow to read and write structured data files with basically any nested format, which makes NETCDF ideal for representing the data characterizing SMS++, as discussed below.

3 General SMS++ file structure

In SMS++, files can describe two different kinds of objects:

1. Block objects, representing the data characterizing a specific instance of a (structured) optimization problem to be solved;
2. Configuration objects, representing parameters of the solution process, either pertaining to a Block (say, which of the different available formulations of the optimization problem should be used), or to a Solver used to actually perform the optimization process.

Both objects have `serialize()` and `deserialize()` methods allowing to save the current state to a given NETCDF *file/group*, as well as *factories* allowing to create a new object from scratch out of a given NETCDF *file/group*.

The issue of SMS++ is that an optimization problem is represented by a Block, which can have any number of sub-Block (recursively) representing parts of the problem with a specific structure. Yet, “specific structure” here may mean that the enclosing Block can make some assumptions on what kind of sub-Block it contains, but it may not know exactly their type. In C++ parlance, Block is an abstract base class, and specific optimization problems are represented by specific derived classes (:Block). A given :Block can make assumption on the type of its sub-Block, but there can be many different variants of it, represented by different derived classes from the same base one. Thus, clearly the data-reading must be organized in such a way that each :Block is responsible for retrieving that which is necessary to it without making assumptions about the global organization of the file. This does not only apply to the data of the instance, but also to *configuration* options that a :Block may have. In fact, several aspect of the :Block representing a given problem (which formulation is used, if and how variables and constraint of the formulation are dynamically generated, which parts of the primal/dual solution are saved, ...) can be implemented in different ways. Note that these decisions are in principle independent from the specific instance, and therefore are conceptually to be stored in, and retrieved from, a separate container than that holding the data of the Block; this is why the Configuration class is provided. Since Block is a tree-shaped nested data structure, Configuration also have to be such. Clearly, NETCDF perfectly suit this kind of requirement; in particular, the standing assumption for both classes is that

*the data of one object (Block/Configuration) is all to be found in the same NETCDF group;
the data of its “sub-objects” (sub-Block/sub-Configuration) is to be found in child groups of the group.*

Hence, the format of a SMS++ NETCDF file depends on whether it holds Block information, Configuration information, or both. For the latter, there are actually two separate kind of Configuration information that can be attached to a :Block. The first describes the above-mentioned options that are inherent to the :Block itself, which are described by the specific derived class BlockConfig of Configuration. However, a :Block need be solved, which is done by Solver objects; each Block can have an arbitrary number of Solver “registered” with it. A Solver can be an arbitrarily complex solution process; in particular, the structure of Block immediately implies that one can register Solver with any sub-Block, which is actually useful e.g. to decomposition algorithms. Hence, not only each :Solver can have an arbitrarily large set of algorithmic configuration options; an arbitrarily large set of Solver can be attached to a Block and to all of its sub-Block, recursively. Thus, the different derived class BlockSolverConfig of Configuration is provided to describe all this information. Note, again, that this is conceptually separate from both the specific instance encoded in the :Block and the choice of the Block-specific options encoded in the BlockConfig, which justifies why a separate container is used.

To allow holding all this different information, a SMS++ NETCDF file has the following general structure. It must contain (in the `netCDF::NcFile` object, which is also a `netCDF::NcGroup`) an *attribute* “SMS++_file_type” of `int` type which specifies which of three different kinds of file it is. The values are encoded in the `enum smspp_netCDF_file_type` defined in `SMSTypedefs.h`, as follows:

- `eProbFile = 0`: the *file* (which is also a *group*) has any number of child *groups* with names “Prob_0”, “Prob_1”, ... In turn, each child *group* has *exactly three* child *groups* with names “Block”, “BlockConfig” and “BlockSolver”, respectively. The first is intended to contain the serialization of a :Block, the second the serialization of a :BlockConfig of the same :Block, and the third the serialization of a :BlockSolverConfig of the same :Block, although any of the three can in principle be empty. If any of the child is not empty, it *must* necessarily contain all the information necessary to reconstruct the specific instance.

- eBlockFile = 1: the *file* (which is also a *group*) has any number of child *groups* with names “Block_0”, “Block_1”, ... Each child *group* contains the serialization of a :Block.
- eConfigFile = 2: the *file* (which is also a *group*) has any number of child *groups* with names “Config_0”, “Config_1”, ... Each child *group* contains the serialization of a :Configuration. Note that no distinction is required between a :BlockConfig and a :BlockSolverConfig, since they are both derived from Configuration.

Note that (some variants of) the `deserialize()` methods of Block/ Configuration allow to directly specify which of the different objects stored in the file to read via its index i in the “name”.

To completely describe the file formats we now have to describe the format of the NETCDF *group* as far as the base Block and Configuration (for the latter, actually both sub-classes BlockConfig and BlockSolverConfig) are concerned, i.e., the (possibly, small) part of the contents that do not depend on the specific derived class.

3.1 Block description

The only content that any NETCDF *group* describing a :Block must necessarily have is a string *attribute* with name “type” containing the classname of the object. This must be exact, e.g., as returned by the protected virtual method `name()`, since it is used in the *factory* when deserializing the object. Note that template classes derived from Block are allowed, with the standard notation “class_name<template-paramaters>”.

3.2 Configuration description

The only content that any NETCDF *group* describing a :Configuration must necessarily have is a string *attribute* with name “type” containing the classname of the object. This must be exact, e.g., as returned by the protected virtual method `name()`, since it is used in the *factory* when deserializing the object. Note that template classes derived from Configuration are allowed, with the standard notation “class_name<template-paramaters>”; see for instance the template SimpleConfiguration<> class.

3.2.1 BlockConfig description

Besides the mandatory “type” attribute of any :Configuration, a SMS++ NETCDF *group* for a BlockConfig can contain the following:

- the *attribute* “name” of string type containing the “name” of the Block;
- the *group* “static_constraints” containing a Configuration object for the static constraints of the Block;
- the *group* “dynamic_constraints ” containing a Configuration object for the dynamic constraints of the Block;
- the *group* “static_variables” containing a Configuration object for the static variables of the Block;
- the *group* “dynamic_variables” containing a Configuration object for the dynamic variables of the Block;
- the *group* “objective” containing a Configuration object for the objective of the Block;
- the *group* “is_feasible” containing a Configuration object for the `is_feasible()` method of the Block;
- the *group* “is_optimal” containing a Configuration object for the `is_optimal()` method of the Block;
- the *group* “solution” containing a Configuration object for the methods of the Block dealing with solutions (`get_Solution()` and `map_[back/forward]_solution()`);
- the *group* “extra” containing a Configuration object, which has no direct use in the base Block class, but is added so that derived classes can put there any configuration information without having to define further derived classes form BlockConfig (which, however, they can still do if they want);

- the *dimension* “n_sub_Block” containing the number of BlockConfig descriptions for the sub-Block of the current :Block;
- with n being the size of n_sub_Block, n groups, with name “sub-BlockConfig_” i ” for all $i = 0, \dots, n - 1$, containing each the description of a BlockConfig for one of the sub-Block of the current :Block.

Of all these, only the “name” *attribute* and the “n_sub_Block” *dimension* are mandatory (they must exist, although they may be empty/zero). All other groups may not exist, in which case the corresponding field of the class is filled with a nullptr, indicating that the “default” configuration (whatever that may mean for the :Block in question) have to be used. Note that that the matching between the sub-BlockConfig and the sub-Block is positional: the BlockConfig found in the group “sub-BlockConfig_” i ” is that for the i -th sub-Block. Note that the vector of sub-BlockConfig is allowed to be of different size than the number of sub-Block; if it is larger any extra BlockConfig is simply ignored, if it shorted then all missing sub-BlockConfig are treated as nullptr (default configuration).

3.2.2 BlockSolverConfig description

Besides the mandatory “type” attribute of any :Configuration, a SMS++ NETCDF *group* for a BlockSolverConfig can contain the following:

- the *dimension* “n_SolverConfig” containing the number of Solver that are to be attached to this Block, and therefore the number of their :Configuration objects;
- the *variable* “SolverNames”, of type string and indexed over the dimension “n_SolverConfig”; the i -th entry of the variable is assumed to contain the classname of a :Solver object to be attached to the Block (this must be exact, e.g., as returned by the protected virtual method name(), since it is used in the *factory* when creating the object);
- with n being the size of n_SolverConfig, n groups, with name “SolverConfig_” i ” for all $i = 0, \dots, n - 1$, containing each the description of a :Configuration object for the i -th :Solver;
- the *dimension* “n_BlockSolverConfig” containing containing the number of BlockSolverConfig descriptions for the sub-Block of the current :Block;
- with m being the size of n_BlockSolverConfig, m groups, with name “BlockSolverConfig_” i ” for all $i = 0, \dots, m - 1$, containing each the description of a BlockSolverConfig for one of the sub-Block of the current :Block.

Of all these, only the “n_SolverConfig” and “n_BlockSolverConfig” *dimensions* and the “SolverNames” *variable* are mandatory; the individual *groups* may not exist if the corresponding dimension is 0. Note that that the matching between the sub-BlockSolverConfig and the sub-Block is positional: the BlockSolverConfig found in the group “BlockSolverConfig_” i ” is that for the i -th sub-Block. Note that the vector of sub-BlockSolverConfig is allowed to be of different size than the number of sub-Block; if it is larger any extra BlockSolverConfig is simply ignored, if it shorted then all missing sub-BlockConfig are treated as nullptr (default configuration).

Note that the :Configuration objects to be found in the *groups* “SolverConfig_” i ” are not “any” :Configuration; rather, they are assumed to be of the specific type ComputeConfig, whose format is described in the next subsection.

3.2.3 ComputeConfig description

The ComputeConfig class (defined in ThinComputeInterface.h) is a class of :Configuration objects specifically tailored for representing sets of algorithmic parameters of solution algorithms, like the ones presumably required by a :Solver. Besides the mandatory “type” attribute of any :Configuration, a SMS++ NETCDF *group* for a ComputeConfig can contain the following:

- the *attribute* “diff” of int type containing the value for the f_diff field of the ComputeConfig (basically, a bool telling if the information in it has to be taken as “the configuration to be set” or as “the changes to be made from the current configuration”);
- the *dimension* “num_int_par” containing the number of int parameters;

- the *variable* “int_par_names”, of type string and indexed over the dimension “num_int_par”; the i -th entry of the variable is assumed to contain the string name of an int parameter;
- the *variable* “int_par_vals”, of type int and indexed over the dimension “num_int_par”; the i -th entry of the variable is assumed to contain the value of the int parameter whose string name is to be found in the i -th entry of “int_par_names”;
- the *dimension* “num_dbl_par” containing the number of double parameters;
- the *variable* “dbl_par_names”, of type string and indexed over the dimension “num_dbl_par”; the i -th entry of the variable is assumed to contain the string name of a double parameter;
- the *variable* “dbl_par_vals”, of type double and indexed over the dimension “num_dbl_par”; the i -th entry of the variable is assumed to contain the value of the double parameter whose string name is to be found in the i -th entry of “dbl_par_names”;
- the *dimension* “num_str_par” containing the number of string parameters;
- the *variable* “str_par_names”, of type string and indexed over the dimension “num_str_par”; the i -th entry of the variable is assumed to contain the string name of a string parameter;
- the *variable* “str_par_vals”, of type string and indexed over the dimension “num_str_par”; the i -th entry of the variable is assumed to contain the value of the string parameter whose string name is to be found in the i -th entry of “str_par_names”;
- the *group* “extra” containing a Configuration object, which has no direct use in the base ComputeConfig class, but is added so that derived classes can put there any configuration information without having to define further derived classes from ComputeConfig (which, however, they can still do if they want).

The three *dimensions* “num_*_par” are mandatory. If any of these is zero, the corresponding variables “*_par_names” and “*_par_vals” may not be defined. The “extra” *group* may not exist, in which case the corresponding Configuration object is set to a nullptr.

4 File formats of existing :Block

This Section collects the description of the NETCDF file formats for all the implemented :Block developed for PLAN4RES. As such, the section will be continuously updated over the course of the project as new :Block will be added, although as usual changes to existing input formats will be kept to a minimum, and always agreed with the partners involved with producing them before being put in production.

4.1 SimpleMILPBlock

SimpleMILPBlock is a simple classe, only having the “abstract representation”, for describing Mixed-Integer Linear Programs of the generic form

$$\min\{\bar{c} + cx : \underline{b} \leq Ax \leq \bar{b}, \quad l \leq x \leq u\}$$

where $c, l, u \in \mathbb{R}^n$, $\underline{b}, \bar{b} \in \mathbb{R}^m$, $A \in \mathbb{R}^{m \times n}$, and $c^0 \in \mathbb{R}$. Actually, entries of $\underline{b}$ and l can be $-\infty$, and entries of $\bar{b}$ and u can be $+\infty$. The coefficient matrix A is usually very sparse, which justifies the choice of a standard sparse (row-wise) format to represent it.

Besides the mandatory “type” attribute of any :Block, the group will contain the following:

- the *dimension* “num_vars” containing the number n of Variable (columns in the matrix A) of the SimpleMILPBlock;
- the *dimension* “num_constraints” containing the number m of Constraint (rows in the matrix A) of the SimpleMILPBlock;
- the *dimension* “nzcount” containing the total number of nonzero coefficients in the Constraint of the SimpleMILPBlock;
- the *variable* “rmatval”, of type double and indexed over the *dimension* “nzcount”, that contains the nonzeros in the coefficient matrix A of the MILP, arranged row-wise (see rmatbeg);
- the *variable* “rmatind”, of type UInt64 and indexed over the *dimension* “nzcount”, that contains the column indices of the nonzeros in the coefficient matrix A of the MILP, arranged row-wise; that is, the i -th entry of rmatind indicates to which variable (column) the (allegedly, nonzero) coefficient to be found in the i -th entry of rmatval refers;
- the *variable* “rmatbeg”, of type UInt64 and indexed over the *dimension* “num_constraints”, that contains the indices into the rmatval/rmatind vectors where each constraint (row of the coefficient matrix A of the MILP) begins: all the nonzeros corresponding to constraint $h = 0, \dots, \text{num_constraints} - 1$ are to be found in rmatval[rmatbeg[h]], ..., rmatval[rmatbeg[$h + 1$] - 1], with their column (variable) indices to be found in the corresponding entries of rmatind[];
- the *variable* “x_lb”, of type double and indexed over the *dimension* “num_vars”, whose i -th entry is the (possibly, $-\infty$) lower bound on the feasible value of the i -th variable (the vector l);
- the *variable* “x_ub”, of type double and indexed over the *dimension* “num_vars”, whose i -th entry is the (possibly, $+\infty$) upper bound on the feasible value of the i -th variable (the vector u);
- the *variable* “row_lb”, of type double and indexed over the *dimension* “num_constraints”, whose i -th entry is the (possibly, $-\infty$) lower bound on the feasible value that the linear expression of the i -th constraint (as specified by the coefficients in rmatval, rmatind, and rmatbeg) can take (the vector $\underline{b}$);
- the *variable* “row_ub”, of type double and indexed over the *dimension* “num_constraints”, whose i -th entry is the (possibly, $+\infty$) upper bound on the feasible value that the linear expression of the i -th constraint (as specified by the coefficients in rmatval, rmatind, and rmatbeg) can take (the vector $\bar{b}$);
- the *variable* “obj”, of type double and indexed over the *dimension* “num_vars”, whose i -th entry is the coefficient of the i -th variable in the linear objective function of the MILP (the vector c);
- the scalar *variable* “obj_offset”, of type double (and, since it is a scalar, not indexed over any dimension) giving the fixed offset in the linear objective function of the MILP (the constant $\bar{c}$).

All these are mandatory.

4.2 StructuredMILPBlock

While SimpleMILPBlock is a “leaf” Block just representing a generic MILP, StructuredMILPBlock derives from SimpleMILPBlock and extends it to represent “structured” MILP of the format

$$\begin{aligned} \min \bar{c} + c^0 x^0 + \sum_{h=1}^k c^h x^h \\ x^h \in X^h \quad h = 1, \dots, k \\ \underline{b}^0 \leq A^0 x^0 \leq \bar{b}^0, \quad l^0 \leq x^0 \leq u^0 \\ r \leq B^0 x^0 + \sum_{h=1}^k B^h x^h \leq \bar{r} \end{aligned}$$

where X^h denotes the feasible region of the h -th sub-Block of the StructuredMILPBlock, which must be a SimpleMILPBlock, x^h its Variable, and c^h the coefficients of its objective. Thus, a StructuredMILPBlock has its own MILP constraints on its own variable x^0 , represented exactly as in SimpleMILPBlock (indeed, a StructuredMILPBlock is a SimpleMILPBlock), plus the constraints of an arbitrary numbers of SimpleMILPBlock, plus constraints “linking” the variable x^0 of the “father” with those of the “sons”. Note that the “sons” of a StructuredMILPBlock are SimpleMILPBlock, which means they can actually be other StructuredMILPBlock. Thus, StructuredMILPBlock allows to represent any block-structured MILP *provided that the linking constraints only link variables of the father to variables of its immediate sons, and not to variables of further descendants.*

Besides the mandatory “type” attribute of any :Block, and whatever has to be found in the netCDF::NcGroup of a SimpleMILPBlock (since a StructuredMILPBlock is a SimpleMILPBlock), the group should contain the following:

- the *dimension* “linking_num_constraints” containing the number of “linking” Constraint (rows of the matrices B^h) of the StructuredMILPBlock;
- the *dimension* “linking_nzcount” containing the total number of nonzero coefficients in the “linking” Constraint (the matrices B^h) of the StructuredMILPBlock;
- the *variable* “linking_rmatval”, of type double and indexed over the *dimension* “linking_nzcount”, that contains the nonzeros in the coefficient matrix of the “linking” Constraint (the matrices B^h), arranged row-wise (see linking_rmatbeg);
- the *variable* “linking_rmatind”, of type UInt64 and indexed over the *dimension* “linking_nzcount”, that contains the column indices of the nonzeros in the coefficient matrix of the “linking” Constraint (the matrices B^h), arranged row-wise; this has to be used together with linking_rmatblk, see below;
- the *variable* “linking_rmatblk”, of type UInt64 and indexed over the *dimension* “linking_nzcount”, that contains the block indices of the variables corresponding to nonzeros in the coefficient matrix of the “linking” Constraint (the matrices B^h); that is, the i -th entry of linking_rmatblk indicates the index h of the Block to which the Variable belongs, with $h = 0$ used for the “father” StructuredMILPBlock and $h = 1, 2, \dots$ used for the Variable of the 1st, 2nd, ... of its SimpleMILPBlock sons, while the i -th entry of linking_rmatind is the index of the Variable in the h -th Block;
- the *variable* “linking_rmatbeg”, of type UInt64 and indexed over the *dimension* “linking_num_constraints”, that contains the indices into the linking_rmatval/linking_rmatind vectors where each “linking” Constraint (row of the matrices B^h) begins: all the nonzeros corresponding to constraint $h = 0, \dots, \text{linking_num_constraints} - 1$ are to be found in linking_rmatval[linking_rmatbeg[h]], ..., linking_rmatval[linking_rmatbeg[$h + 1$] - 1], with their column (variable) and Block indices to be found in the corresponding entries of linking_rmatind and linking_rmatblk, respectively;
- the *variable* “linking_row_lb”, of type double and indexed over the *dimension* “linking_num_constraints”, whose i -th entry is the (possibly, $-\infty$) lower bound on the feasible value that the linear expression of the i -th “linking” Constraint (as specified by the coefficients in linking_rmatval, linking_rmatind, linking_rmatbeg, and linking_rmatblk) can take;
- the *variable* “linking_row_ub”, of type double and indexed over the *dimension* “linking_num_constraints”, whose i -th entry is the (possibly, $+\infty$) upper bound on the feasible value that the linear expression of the i -th

“linking” Constraint (as specified by the coefficients in `linking_rmatval`, `linking_rmatind`, `linking_rmatbeg`, and `linking_rmatblk`) can take.

All these are mandatory.

4.3 MCFBlock

The `MCFBlock` class implements the `Block` concept for the (linear) Min-Cost Flow (MCF) problem.

The data of the problem consist of a (directed) graph $G = (N, A)$ with $n = |N|$ nodes and $m = |A|$ (directed) arcs. Each node i has a deficit $b[i]$, i.e., the amount of flow that is produced/consumed by the node: source nodes (which produce flow) have negative deficits and sink nodes (which consume flow) have positive deficits. Each arc (i, j) has an upper capacity $U[i, j]$ and a linear cost coefficient $C[i, j]$. Flow variables $X[i, j]$ represents the amount of flow to be sent on arc (i, j) . Parallel arcs, i.e., multiple copies of the same arc (i, j) are in general allowed; it is expected that they have different costs (for otherwise they can be merged into a unique arc), but this is not strictly enforced. Multiple copies of some arc (i, j) can be seen as “total” flow cost on that arc being a piecewise-linear convex function. The formulation of the problem is:

$$\begin{aligned} \min \quad & \sum_{(i,j) \in A} C[i, j] X[i, j] \\ \sum_{(j,i) \in A} X[j, i] - \sum_{(i,j) \in A} X[i, j] &= b[i] & i \in N \\ 0 \leq X[i, j] &\leq U[i, j] & (i, j) \in A \end{aligned}$$

The n equations are the flow conservation constraints and the $2m$ inequalities are the flow nonnegativity and capacity constraints. At least one of the flow conservation constraints is redundant, as the demands must be balanced ($\sum_{i \in N} b[i] = 0$); indeed, exactly $n - \text{ConnectedComponents}(G)$ flow conservation constraints are redundant, as demands must be balanced in each connected component of G .

Besides the mandatory “type” attribute of any `:Block`, the *group* representing the `MCFBlock` should contain the following:

- the *dimension* “NNodes” containing the number of nodes in the graph (n);
- the *dimension* “NArcs” containing the number of arcs in the graph (m);
- the *variable* “C”, of type `double` and indexed over the dimension “NArcs”; the i -th entry of the variable is assumed to contain the upper capacity of the i -th arc in the graph, that whose starting node and ending node are specified by the *variable* “SN” and “EN” (below);
- the *variable* “U”, of type `double` and indexed over the dimension “NArcs”; the i -th entry of the variable is assumed to contain the cost of the i -th arc in the graph (the lower capacity being fixed to 0), that whose starting node and ending node are specified by the *variable* “SN” and “EN” (below);
- the *variable* “B”, of type `double` and indexed over the dimension “NNodes”; the i -th entry of the variable is assumed to contain the deficit of the i -th node in the graph (note that node names here go from 0 to $n - 1$);
- the *variable* “SN”, of type `int` and indexed over the dimension “NArcs”; the i -th entry of the variable is assumed to contain the starting node of the i -th arc in the graph (note that node names here go from 1 to n , they are shifted by 1 w.r.t. to the indices used in the “B” variable);
- the *variable* “EN”, of type `int` and indexed over the dimension “NArcs”; the i -th entry of the variable is assumed to contain the ending node of the i -th arc in the graph (note that node names here go from 1 to n , they are shifted by 1 w.r.t. to the indices used in the “B” variable).

The two *dimensions* “NNodes” and “NArcs” are mandatory, such as are the two *variables* “SN” and “EN”. The three other *variables* are optional. If “C” is missing, all arc costs are assumed to be 0. If “U” is missing, all arc capacities are assumed to be infinite. If “B” is missing, all node deficits are assumed to be 0.

5 BlockConfig for existing :Block

As described in §3.2.1, each Block can have several Configuration parameters, which can be dealt with by a BlockConfig object. It makes therefore sense to collect here all possible Configuration parameters for existing :Block, since these may end up in a SMS++ NETCDF file.

5.1 Parameters of MCFBlock

The MCFBlock class defines the following Configuration parameters:

- `f_static_constraints.Configuration`. The static constraint of a MCF are the flow conservation equations and the bound constraints. The latter are typically implemented via a `std::vector<LB0Constraint>` with exactly m entries. However, If *all* the upper bound are $+\infty$, it is possible to skip the `std::vector<LB0Constraint>` entirely and just use the fact that the ColVariable can be defined to be non-negative. If `f_static_constraints` is set, it is a `SimpleConfiguration<int>` whose `f_value` is not 0, the bound constraints are not implemented if it is possible to do so; otherwise they are always implemented. Note that this makes it impossible to change any upper bound.
- `f_is_feasible.Configuration`. If `f_is_feasible.Configuration` is a `SimpleConfiguration<double>`, its `f_value` is taken as the parameter for deciding what “approximately feasible” exactly means while checking satisfaction of both flow conservation constraint and flow upper/lower bounds; the value is taken as the *relative* tolerance for those checks.
- `f_is_optimal.Configuration`. Checking optimality of a primal-dual solution of the MCF problems means checking that both are approximately feasible, and that they approximately satisfy the Complementary Slackness Conditions. This requires two parameters for deciding what “approximately feasible” means, one for the primal (ϵ_f) and one for the dual (ϵ_c). If `f_is_optimal.Configuration` is a `SimpleConfiguration<std::pair<double, double>>`, then $\epsilon_c = \text{f_value.first}$ and $\epsilon_f = \text{f_value.second}$.
- `f_solution.Configuration`. For all the methods dealing with solutions, i.e., `get_Solution()` and `map_[back/forward]` if `f_solution.Configuration` is a `SimpleConfiguration<int>`, then its `f_value` decides what is mapped, with the following encoding: 1 means “only map the primal solution”, 2 means “only map the dual solution”, everything else (e.g., 0) means “map everything”.

6 ComputeConfig for existing :Solver

As described in §3.2.3, each Block can have several ComputeConfig parameters, which can be dealt with by a BlockSolverConfig object. It makes therefore sense to collect here all possible ComputeConfig parameters for the existing objects that define actual values for their algorithmic parameters, since these may end up in a SMS++ NETCDF file.

6.1 Standard parameters in Solver

The base Solver class defines a set of “general” algorithmic parameters, described in the following Table.

name	title	description
intMaxIter	int	The algorithmic parameter for setting the maximum number of iterations that the next call to solve() is allowed to execute for trying to solve the Block. The concept of “what exactly an iteration is” is clearly Solver-dependent, and the user of the Solver need supposedly be aware of which concrete :Solver it is actually using to be able to sensibly set this parameter; however, because most Solver will actually be iterative processes, it makes sense to offer support for this notion in the base class.
intMaxSol	int	The algorithmic parameter for setting the maximum number of different solutions to the Block that the Solver should attempt to obtain and store. Mathematical models can have (very) many solutions: an objective function precisely helps in selecting among them, but even that may not be enough to narrow the choice down to a single solution (multiple optima may exist, or quasi-optimal solutions may also be sought for for any number of reasons). On the other hand, a solution can be a “big” object, and storing it may be costly. It may be therefore helpful for a Solver to know beforehand how many different solutions the user would like to get. Among the reasonable values for this parameter, 1 says “I don’t care of multiple solutions, give me only the best one”, and 0 says “I don’t care of solutions at all, just tell me if there is any, and what its value is”. The default is 1.
intLogVerb	int	An integer parameter dictating how “verbose” the log of the Solver has to be. The specific meaning of each value is Solver-dependent, but it is intended that 0 means “no log at all”, and increasing values correspond to increasing verbosity. The default value is 0 (no log).
dblMaxTime	dbl	The algorithmic parameter for setting the maximum time limit that the next call to solve() can expend in trying to solve the Block. The value is assumed to be in seconds, and it’s a double (so both very quick and very slow solvers are supported). The base Solver class does not explicitly distinguish between “wall-clock time” and “CPU time”, which may be rather different especially in a parallel environment, but this concept can be easily added by derived classes.
dblRelAcc	dbl	The algorithmic parameter for setting the <i>relative</i> accuracy required to the solution of the Block. This is geared towards single-objective optimization problems, and it is defined as follows: a solution for a <i>minimization</i> problem has a relative accuracy of ϵ if a feasible (to within the required tolerance for each Block) solution has been obtained, an upper bound ub on its objective function value has been found, a lower bound lb on the optimal value of the problem has been found, and $ub - lb \leq \epsilon \max(lb , 1)$. The roles of ub and lb are suitably reversed for a maximization problem. The default is $1e-6$.
dblAbsAcc	dbl	The algorithmic parameter for setting the <i>absolute</i> accuracy required to the solution of the model. This is geared towards single-objective optimization problems, and it is defined as follows: a solution for a <i>minimization</i> problem has a absolute accuracy of ϵ if a feasible (to within the required tolerance for each Block) solution has been obtained, an upper bound ub on its objective function value has been found, a lower bound lb on the optimal value of the problem has been found, and $ub - lb \leq \epsilon$. The roles of ub and lb are suitably reversed for a maximization problem. The default is $+\infty$, which is intended to mean that the only working accuracy is the relative one.

dblUpCutOff	dbl	The algorithmic parameter for setting the <i>upper cut off</i> of the solution process. This is a value basically telling the Solver “when enough is enough”. In particular, if the Solver obtains a certified lower bound lb on the optimal value such that $lb \geq \varepsilon$ with the provided parameter ε , then it can stop. This is a certificate that the optimal value is at least ε . For a maximization problem the condition says: I actually needed a solution with objective function value at least as good as (i.e., larger than) ε , now that I have found one the problem is as good as solved to me. Hence this parameter is analogous to the maximum absolute accuracy dblAbsAcc, but “weaker” in that it does not require any upper bound to work. For a minimization problem, instead, the condition says: I actually needed a solution with objective function value at least as good as (i.e., smaller than) ε , now that I have know for sure that this is never going to happen the problem is as good as unfeasible to me. The default is $+\infty$.
dblLwCutOff	dbl	The algorithmic parameter for setting the <i>lower cut off</i> of the solution process. This is a value basically telling the Solver “when enough is enough”. In particular, if the Solver obtains a certified upper bound ub on the optimal value such that $ub \leq \varepsilon$ with the provided parameter ε , then it can stop. This is a certificate that the optimal value is at most ε . For a minimization problem the condition says: I actually needed a solution with objective function value at least as good as (i.e., smaller than) ε , now that I have found one the problem is as good as solved to me. Hence this parameter is analogous to the maximum absolute accuracy dblAbsAcc, but “weaker” in that it does not require any lower bound to work. For a maximization problem, instead, the condition says: I actually needed a solution with objective function value at least as good as (i.e., larger than) ε , now that I have know for sure that this is never going to happen the problem is as good as unfeasible to me. The default is $-\infty$.
dblRAccSol	dbl	The algorithmic parameter for setting the relative accuracy of the accepted solutions. It instructs the Solver not to even consider a solution among the ones to be reported intMaxSol if its objective function value is “too” bad. For a minimization problem, the objective function value of a feasible solution provides an upper bound ub on the optimal value. Assuming a lower bound lb on the optimal value of the problem has been found, a solution is deemed acceptable with the provided parameter ε if $ub - lb \leq \varepsilon \max(ub , lb , 1)$. Note that if no lb is available, the above formula can be replaced with $ub - f_{best} \leq \varepsilon \max(ub , f_{best} , 1)$, where f_{best} is the value of the best (with smallest objective value) solution found so far. The roles of ub and lb are suitably reversed for a maximization problem. The default is $+\infty$.
dblAAccSol	dbl	Similar to dblRAccSol but for an <i>absolute</i> accuracy; that is, a solution is deemed acceptable with the provided parameter ε if $ub - lb \leq \varepsilon$ or $ub - f_{best} \leq \varepsilon$ with the same notation as in dblRAccSol and the same provisions about the case of a maximization problem. The default is $+\infty$.
dblFAccSol	dbl	The algorithmic parameter for setting the maximum relative allowed violation of constraints. Whenever the Solver is incapable of finding feasible solutions (maybe because there is none), it may still be useful that it returns the “least unfeasible” ones. This parameter instructs the Solver not to even consider a solution among the ones to be reported (see intMaxSol) if its violation is “too” bad. The actual meaning of this parameter may be Solver-dependent, but it can be thought to work as the “relative constraint violation”. A setting of 0 may be taken as a way to tell the Solver not to bother to produce unfeasible solutions at all, which is why this is the default value of the parameter.

6.2 Standard parameters in CDASolver

The CDASolver class extends the parameters of Solver with the ones described in the following Table.

name	title	description
------	-------	-------------

intMaxDSol	int	The algorithmic parameter for setting the maximum number of different <i>dual</i> solutions that the Solver should attempt to obtain and store. Since dual solutions can be a “big” objects (in some cases, much bigger than primal ones), storing them may be costly. It may be therefore helpful for a CDASolver to know beforehand how many different dual solutions the user would like to get. Among the reasonable values for this parameter, 1 says “I don’t care of multiple dual solutions, give me only the best one”, and 0 says “I don’t care of dual solutions at all, just tell me if there is any, and what its value is”. The default is 1.
dblRAccDSol	dbl	The algorithmic parameter for setting the relative accuracy of the accepted <i>dual</i> solutions. It instructs the CDASolver not to even consider a solution among the ones to be reported (see intMaxDSol) if its objective function value is “too” bad. If the original problem is a minimization one, its dual is a maximization one. Hence, the objective function value of a dual solution provides a lower bound lb on the optimal dual value (which may, or may not, be equal to that of the primal problem). Assuming an upper bound ub on the optimal value of the <i>dual</i> problem has been found (note that this can be obtained by means of the objective value of a feasible <i>primal</i> solution), a solution is deemed acceptable with the provided parameter ε if $ub - lb \leq \varepsilon \max(ub , lb , 1)$. Note that if no ub is available, the above formula can be replaced with $f_{best} - lb \leq \varepsilon \max(f_{best} , lb , 1)$ where f_{best} is the value of the best (with largest objective value) solution found so far. The roles of ub and lb are suitably reversed if the primal is a maximization problem, so that the dual is a minimization one. The default is $+\infty$.
dblAAccDSol	dbl	Similar to dblRAccDSol but for an <i>absolute</i> accuracy; that is, a dual solution is deemed acceptable with the provided parameter ε if $ub - lb \leq \varepsilon$ or $f_{best} - lb \leq \varepsilon$ with the same notation as in dblRAccDSol and the same provisions about the case of a maximization problem. The default is $+\infty$.
dblFAccDSol	dbl	The algorithmic parameter for setting the maximum relative allowed violation of <i>dual</i> constraints, assuming of course something like that exists in the specific dual that is being dealt with. Whenever the CDASolver is incapable of finding feasible dual solutions (maybe because there is none), it may still be useful that it returns the “least unfeasible” ones. This parameter instructs the CDASolver not to even consider a solution among the ones to be reported (see intMaxDSol) if its violation is “too” bad. The actual meaning of this parameter is necessarily be CDASolver-dependent; intuitively, it may be thought to work as the “relative constraint violation” <code>feas_epsilon</code> of Block if the concept of “dual constraint” is applicable. A setting of 0 may be taken as a way to tell the CDASolver not to bother to produce unfeasible solutions at all, which is why this is the default value of the parameter.

6.3 Standard parameters in Function

The base Function class defines a set of “general” algorithmic parameters, described in the following Table.

name	title	description
intMaxIter	int	The algorithmic parameter for setting the maximum number of iterations in the next call to <code>compute()</code> . The concept of “what exactly an iteration is” is clearly dependent on the Function, and it may not even make sense for all Function. However, some Function will actually be iterative processes, and therefore it makes sense to offer support for this notion in the base class. The default is $+\infty = \text{no limits}$.
dblMaxTime	dbl	The parameter for setting the maximum time limit for the next call to <code>compute()</code> . The value is assumed to be in seconds, and it’s a double (so both very fast and very slow computations are supported). The Function class (so far) does not explicitly distinguish between “wall-clock time” and “CPU time”, which may be rather different especially in a parallel environment. The default is $+\infty$.
dblRelAcc	dbl	The parameter for setting the <i>relative</i> accuracy required to the function value. That is, if both an upper bound ub and a lower bound lb on the value have been found, then <code>compute()</code> can stop as soon as $ub - lb \leq \text{dblRelAcc} \max(lb , 1)$. The default is $1e-6$.

dblAbsAcc	dbl	The parameter for setting the <i>absolute</i> accuracy required to the function value. That is, if both an upper bound ub and a lower bound lb on the value have been found, then <code>compute()</code> can stop as soon as $ub - lb \leq \text{dblRelAcc}$. The default is $+\infty$, which is intended to mean that the only working accuracy is the relative one.
dblUpCutOff	dbl	The parameter for setting the “upper cut off” of the computation; that is, if a lower bound lb on the value has been found, then <code>compute()</code> can stop as soon as $lb \geq \text{dblUpCutOff}$. This is a certificate that the value is at least <code>dblUpCutOff</code> . The default is $+\infty$, i.e., no upper cut off.
dblLwCutOff	dbl	The parameter for setting the “lower cut off” of the computation; that is, if an upper bound ub on the value has been found, then <code>compute()</code> can stop as soon as $ub \leq \text{dblLwCutOff}$. This is a certificate that the value is at most <code>dblLwCutOff</code> . The default is $-\infty$, i.e., no lower cut off.

6.4 Standard parameters in C05Function

The C05Function class extends the parameters of Function with the ones described in the following Table.

name	title	description
intLPMaxSz	int	The algorithmic parameter for setting the size of the “local pool”, that is, the maximum number of linearizations that should be stored in the local pool. The default is 1, which corresponds to the fact that the Function can only produce a single linearization at a time (for it is, say, smooth).
intGPMaxSz	int	The algorithmic parameter for setting the size of the “global pool”, that is, the maximum number of linearizations that should be stored in the local pool. The default is 0, which corresponds to the fact that the Function cannot store any linearization (for it is, say, smooth and therefore there is no need to).
dblRAccLin	dbl	A linearization (g, α) computed at the point x is “accurate” if the value of the linearization coincides with the value of the function at x , i.e., $\alpha = f(x)$. In general linearizations that are not “completely accurate” can still be useful: for instance, in the Lagrangian case an ε -optimal solution to the Lagrangian problem gives rise to a valid linearization (g, α) with $\varepsilon \geq f(x) - \alpha$. This can be deemed interesting if ε is “small”, but not if ε is “large”. This parameter instructs the C05Function not to bother reporting (and therefore storing in the “local pool”) any linearization having a relative error with $f(x)$ larger than <code>dblRAccLin</code> . This would generally mean $ f(x) - \alpha \leq \text{dblRAccLin} \max(f(x) , \alpha , 1)$, except that the value $f(x)$ may not be known exactly, with only lower and/or upper bounds on it available. The actual formula therefore depends on what information is actually available: for instance, in the Lagrangian case one knows that $f(x) \geq \alpha$, and therefore typically an upper estimate $ub \geq f(x)$ is used in the formula instead of $f(x)$. The default is 0, i.e., “only perfect linearizations are allowed”.
dblAAccLin	dbl	Similar to <code>dblRAccLin</code> but for an <i>absolute</i> accuracy; that is, a linearization is deemed acceptable if $ f(x) - \alpha \leq \text{dblAAccLin}$, except that the value $f(x)$ may not be known exactly, with only lower and/or upper bounds on it available. The actual formula therefore depends on what information is actually available: for instance, in the Lagrangian case one knows that $f(x) \geq \alpha$, and therefore typically an upper estimate $ub \geq f(x)$ is used in the formula instead of $f(x)$. The default is 0, i.e., “only perfect linearizations are allowed”.

6.5 Parameters of MCFSolver<MCFC>

The MCFSolver<> class derives by CDASolver, and therefore it extends the parameters of with the ones described in the following Table.

name	title	description
kReopt	int	Whether the MCF algorithm (whatever it is) should re-optimize after changes in the data, as opposed to restarting from scratch.

However, `MCFsolver<>` is a template class, whose template parameter actually is a concrete MCF solver object derived from `MCFClass`. Each of these has its own algorithmic parameters, which are described in the next subsections.

6.5.1 Parameters of `MCFsolver<MCFsimplex>`

name	title	description
<code>kAlgPrimal</code>	int	If > 0 it instructs the <code>MCFsimplex</code> solver to use the primal (network) simplex method, otherwise the dual one.
<code>kAlgPricing</code>	int	Selects which pricing rule is used within the algorithm. Possible values are <code>kDantzig = 0</code> (Dantzig's rule, most violated constraint), <code>kFirstEligibleArc = 1</code> (first eligible arc in round-robin), and <code>kCandidateListPivot = 2</code> (Candidate List Pivot Rule).
<code>kNumCandList</code>	int	Parameter to set the number of candidate list for the Candidate List Pivot method.
<code>kHotListSize</code>	int	Parameter to set the size of Hot List for the Candidate List Pivot method.

7 Conclusion

This document is intended as a quick reference manual for anybody who is required to construct SMS++ NETCDF input files. It should be remarked that basically all the `:Block` and `:Configuration` objects are supposed to have an in-memory interface to initialize the data. Thus, a workable way to produce NETCDF input files for those is to use the in-memory interface, and then the `serialize()` method to save it into a file. Yet, NETCDF has interfaces for all primary programming languages, hence files with the required format can be written independently by any SMS++ object.

References

- [1] Unidata “The NETCDF software”, <http://doi.org/10.5065/D6H70CW6>, 2019